BOWLING GREEN STATE UNIVERSITY

DATA 5010 | Learning Activity 1

Pet and Cat

Inheritance

A Python Class Hierarchy with Interactive Input

Bahareh Mohamadtaheri
Data Science

October 12, 2026

Problem Statement

REQUIRED **Pet(name, age) • Cat inherits Pet + breed • 2 distinct objects • print_info() for both**

INPUT

Example entry

Pet

Name: Dobby

Years: 2 Months: 3

Cat

Name: Luna

Years: 1 Months: 6

Breed: Persian

PROCESS

01 **Validate fields**
Names; years ≥ 0; months 0–11

02 **Verify cat breed**
Check against the local list of 106 supported cat breeds.

03 **Create two objects**
Cat inherits Pet and adds breed

04 **Display information**
Inherited print_info(), then breed

OUTPUT

Result after successful entry

Pet Information:

Name: Dobby

Age: 2 years and 3 months

Pet Information:

Name: Luna

Age: 1 year and 6 months

Breed: Persian

Both objects reuse Pet.print_info()

ADDED MENU **1. Enter information** **2. Exit**

Invalid input → correct and retry • Results → return to menu

Key Problems

01 Input validity

What inputs need validation or correction?

-2 years	Negative age
two	Non-numeric age
" "	Whitespace-only name

02 Age representation

How are years and additional months handled?

2 y + 6 mo	Supported input structure
0 y + 8 mo	Input structure
1 y + 15 mo	Rejected; prompt to re-enter

03 Names and breed

How are text values and breed checks handled?

Mr. Whiskers	Multi-word name case
Scottish Fold	Multi-word breed case
Unsupported breed	Show guidance and retry breed only.

04 Interruptions and recovery

How does an incomplete session terminate?

EOF	Error + rerun guidance
Ctrl+C	Cancellation message
Return to menu	No completed result printed

Proposed Solutions: Theory, Algorithms, and Systems

THEORY

Object-Oriented Design

Core Q1

Classes combine data and behavior.

Cat inherits Pet's print_info().

Each object stores its own attribute values.

ALGORITHMS

Input and Recovery Logic

Interactive Extension

1. Validate each field; retry invalid entries.
2. Check years ≥ 0 and months 0–11.
3. Verify breed; correct input or retry / return.

EOF or cancellation: message and exit.

Core Q1 After all fields pass: create objects and print.

SYSTEMS

Modular Python Program

Core Q1 **Interactive Extension**

Small command-line program with separate modules.

main.py
coordinates the run.

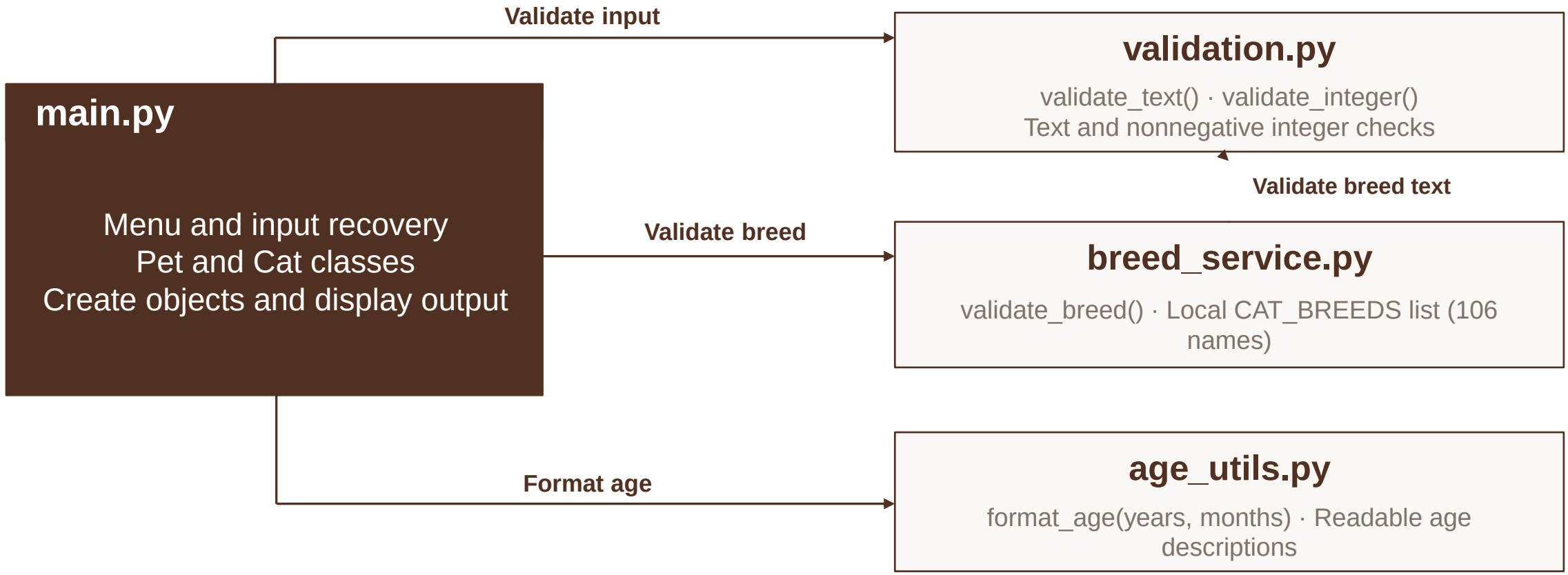
validation.py
Text and integer validation

age_utils.py
Readable age formatting

breed_service.py
A local list supports breed matching.

Pet & Cat: Program Architecture

One coordinating module and three supporting modules



Additional months: 0–11 (checked in main.py) · Local breed list

Pet & Cat: Program Flow and Validation

INPUT VALIDATION

Names / breed text

letters, spaces, hyphens and apostrophes; maximum 30 characters.

Age rules

Years ≥ 0; additional months 0–11. Invalid values are entered again.

Breed matching

Local list of 106 breeds; case and repeated spaces ignored for matching.

START

1 Main menu
Choose 1 to enter information, or 2 to exit

2 Read and validate input
Pet and Cat names, years, and additional months

3 Check breed and create Cat
validate_breed() checks local CAT_BREEDS list

4 Create the separate Pet object
After a valid Cat has been created

5 Display completed information
Inherited print_info() + Cat breed

RETURN TO MAIN MENU

ERRORS & RECOVERY

Invalid menu choice

Show help, then show menu again

Invalid name / age / breed

Show error and retry affected field; excess months retry the age pair.

Interrupted input

EOF → error and exit status 1
Ctrl+C → message and exit

Inside validation.py: Input Validation Flow

How raw input is checked, accepted, or sent back for correction

RAW INPUT from main.py

SHARED PRE-CHECKS — Reject control characters | Remove edge spaces with while loops

TEXT · validate_text(raw, field)

1. **Require nonempty text; maximum 30 characters**
   `if not text / len(text) > MAX_TEXT_LENGTH`
2. **Allow English letters, spaces, - and '**
   `for char in text: if char not in allowed`
3. **Require at least one English letter**
   `has_letter = True; break`

VALID → return text (string)

NUMBER · validate_integer(raw, field)

1. **Require digits; optional leading + or -**
   `for char in digits: if char not in "0123456789"`
2. **Reject more than 32 digits**
   `len(digits) > MAX_NUMBER_DIGITS`
3. **Convert to integer; reject negative values**
   `value = int(text); if value < 0`

VALID → return value (integer)

INVALID AT ANY STEP — raise ValueError → main.py catches it → show guidance → retry that field

Note: main.py separately checks additional months ≤ 11 (excess months repeat the age pair).

Inside breed_service.py: Breed Validation Flow

How breed input moves between main.py, validation.py, and the local supported-breed list

RAW BREED INPUT from main.py

STEP 1 SHARED TEXT CHECK

Cat.__init__() calls breed_service.validate_breed(raw, field)

validate_breed() calls validation.validate_text() first: text format must pass

INSIDE breed_service.py

1. **Create comparison copy**
   comparison_name = " ".join(text.lower().split())
2. **Check local list membership**
   if comparison_name in CAT_BREEDS:

CAT_BREEDS reference

persian · siamese · maine coon
scottish fold · ragdoll · sphynx

Fixed list of 106 lowercase names

Is comparison_name in CAT_BREEDS?

VALID → return original validated text

Cat stores the breed; original case and spacing preserved

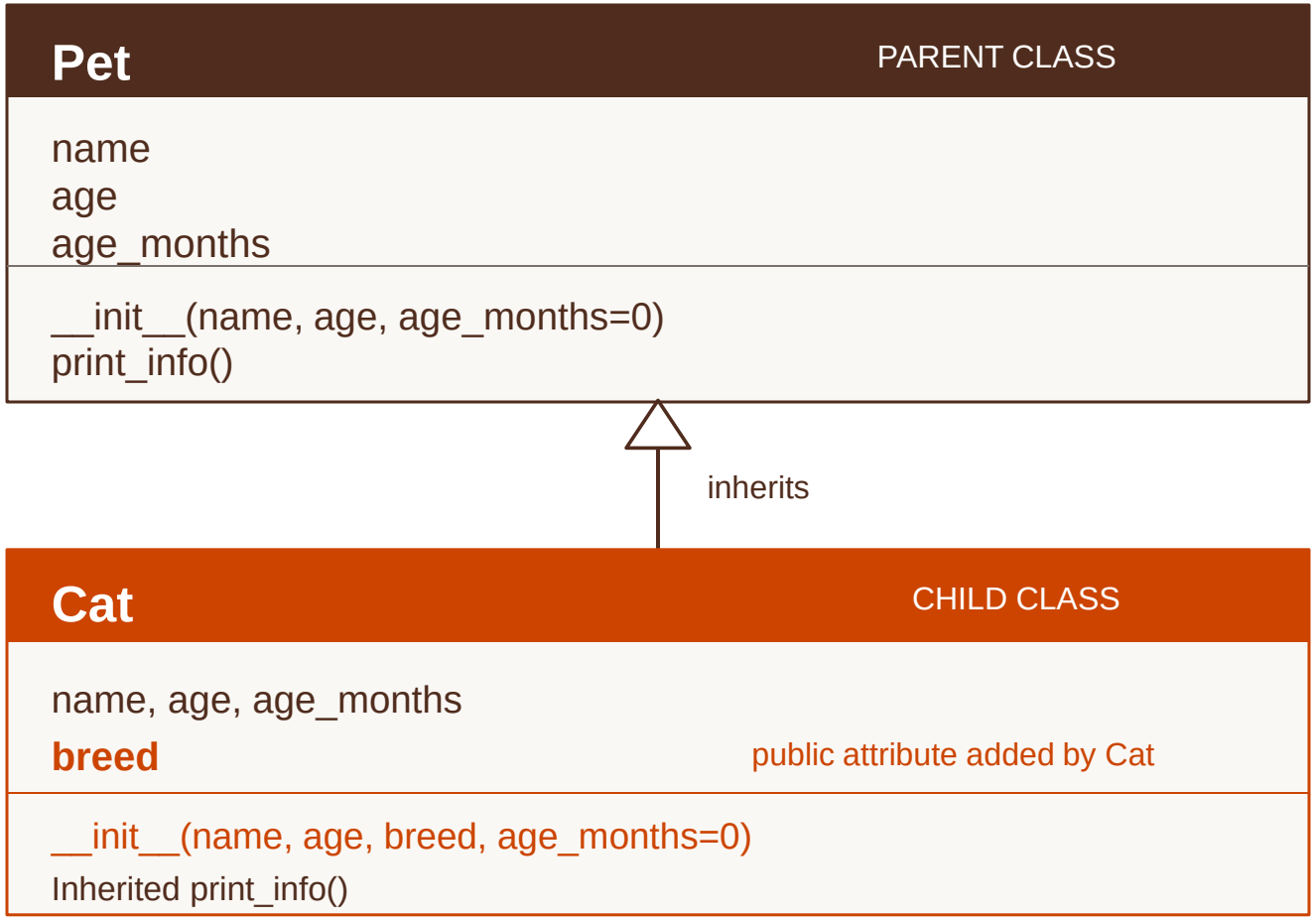
INVALID → raise ValueError

main.py catches error → displays help → retry breed only

CONNECTION: main.py reads input → validation.py checks text → breed_service.py checks supported list

Class Design: Inheritance and Breed Validation

Cat shares Pet's attributes and inherits its printing method



VALIDATE, THEN STORE

Inside Cat.__init__()

```
self.breed = validate_breed(
    breed, "Cat breed"
)
```

Pet & Cat: OOP Concepts in the Code

Pet — constructor and shared printing method

```
class Pet:
    def __init__(self, name, age, age_months=0):
        self.name = name
        self.age = age
        self.age_months = age_months
    def print_info(self):
        print("Pet Information:")
        print(f"   Name: {self.name}")
        print(f"   Age: {format_age(self.age, self.age_months)}")
```

Cat — constructor and breed validation

```
class Cat(Pet):
    def __init__(self, name, age, breed, age_months=0):
        Pet.__init__(self, name, age, age_months)
        self.breed = validate_breed(breed, "Cat breed")
```

print_result() — output calls

```
    pet.print_info()

    print(f"   Breed: {cat.breed}")
```

1 Initialization: __init__()

Sets starting attributes on each object.

2 Instance reference: self

Refers to the current object.

3 Inheritance: Cat(Pet)

Reuses Pet initialization and print_info().

4 Public attribute: breed

Stores the accepted breed on each Cat.

5 Validate, then store

validate_breed() checks the breed before saving.

Current Solution vs. Optional Polymorphism

Same displayed result — two ways to organize printing behavior

CURRENT SOLUTION

Inheritance and reuse

Implemented in main.py

Cat inherits print_info(); breed is printed separately.

```
def print_result(pet, cat):
    pet.print_info()
    cat.print_info()
    print(f"   Breed: {cat.breed}")
```

Both objects use Pet.print_info().

Cat does not override print_info().

OPTIONAL EXTENSION — NOT IMPLEMENTED

Polymorphism

Alternative design

Override print_info() inside Cat:

```
def print_info(self):
    Pet.print_info(self)
    print(f"   Breed: {self.breed}")
```

Then simplify print_result():

```
def print_result(pet, cat):
    for animal in [pet, cat]:
        animal.print_info()
```

Same call; Pet prints name and age,
Cat also prints breed.

Experimental Results: Effectiveness & Efficiency

Validated program behavior, measured performance, and theoretical complexity

EFFECTIVENESS

USER-TESTED

4/4 test categories passed

All four functional groups confirmed successful

PASS	Core functionality & output
PASS	Input and age validation
PASS	Breed validation & correction
PASS	Menu, exit & interruptions

Pass counts reflect categories, not individual test cases.

EFFICIENCY

2.14 µs

Median execution time per operation

THEORETICAL EFFICIENCY

O(1) **Fixed-size core task**

Two objects and a fixed number of checks.

No sorting or growing dataset; breed lookup uses a fixed local list.

Python timeit · 7 repeats × 10,000 runs

Advantages, Limitations & Future Improvements

ADVANTAGES

01 Q1 classes and printing
Creates separate Pet and Cat objects and prints their information using the Q1 approach.

02 Reusable OOP design
Cat inherits from Pet and reuses its print_info() method.

03 Guided input correction
Invalid entries trigger clear messages and prompts to try again.

04 Breed validation at creation
Cat checks the breed before storing it in the public breed attribute.

LIMITATIONS & POSSIBLE IMPROVEMENTS

CURRENT LIMITATION	POSSIBLE IMPROVEMENT
Repeated age entry Months above 11 require entering years again.	**Retry only months** Keep valid years and ask for months again.
Fixed breed list Only 106 supported names are accepted.	**Online breed reference** Fetch breeds through an API with a local backup.
Separate breed printing Cat breed is displayed outside print_info().	**Optional polymorphism** Override print_info() inside Cat.

Thank You!

Questions?

RUN

```
python main.py
```

SOURCE CODE

GitHub: baharem4017/learning-activity-1-pet-cat